Department of Computer Science

Lab Manual

of

DEEP LEARNING

MAI417-3

Class Name: III MScAIML

Master of Science Artifitial Intelligence and Machine Learning

2025-26

Prepared by: Deon Thomas Binny Verified by: Helen K Joy, Shivangi Singh

Department Overview

Department of Computer Science of CHRIST (Deemed to be University) strives to shape outstanding computer professionals with ethical and human values to reshape nation's destiny. The training imparted aims to prepare young minds for the challenging opportunities in the IT industry with a global awareness rooted in the Indian soil, nourished and supported by experts in the field.

Vision

The Department of Computer Science endeavours to imbibe the vision of the University "**Excellence and Service"**. The department is committed to this philosophy which pervades every aspect and functioning of the department.

Mission

"To develop IT professionals with ethical and human values". To accomplish our mission, the department encourages students to apply their acquired knowledge and skills towards professional achievements in their career. The department also moulds the students to be socially responsible and ethically sound.

Introduction to the Programme

Machines are gaining more intelligence to perform human like tasks. Artificial Intelligence has spanned across the world irrespective of domains. MSc (Artificial Intelligence and Machine Learning) will enable to capitalize this wide spectrum of opportunities to the candidates who aspire to master the skill sets with a research bent. The curriculum supports the students to obtain adequate knowledge in the theory of artificial intelligence with hands-on experience in relevant domains with tools and techniques to address the latest demands from the industry. Also, c andidates gain exposure to research models and industry standard application development in specialized domains through guest lectures, seminars, industry offered electives, projects, internships, etc.

Programme Objective

- To acquire in-depth understanding of the theoretical concepts in Artificial Intelligence and Machine Learning
- To gain practical experience in programming tools for Data Engineering, Knowledge Representation, Artificial intelligence, Machine learning, Natural Language Processing and Computer Vision.
- To strengthen the research and development of intelligent applications skills through specialization based real time projects.
- To imbibe quality research and develop solutions to the social issues.

Programme Outcomes:
PO1 : Conduct investigation and develop innovative solutions for real world problems in industry and research establishments related to Artificial Intelligence and Machine Learning
PO2 : Apply programming principles and practices for developing automation solutions to meet future business and society needs.

PO3 : Ability to use or develop the right tools to develop high end intelligent systems
PO4 : Adopt professional and ethical practices in Artificial Intelligence application development
PO5 : Understand the importance and the judicious use of technology for the sustainability of the environment.

MAI417-3– Deep Learning

Total Teaching Hours for Trimester: 75

Max Marks: 100

Credits: 4

Course Description

The course introduces deep learning techniques, focusing on feedforward networks, CNNs, RNNs, and autoencoders. Students will learn regularization methods, optimization for long-term dependencies, and advanced model implementations. Lab exercises provide hands-on experience with Keras and TensorFlow. By the end, students will apply these techniques to real-world problems in computer vision and sequence processing.

Course Objectives

The main objective of this course is to make students comfortable with the tools and techniques required to handle large datasets. Several libraries and datasets publicly available will be used to illustrate the application of these algorithms. This will help students develop the skills required to gain experience in doing independent research and study.

Course Outcomes

Upon successful completion of the course, the student will be able to

CO1: Implement deep feedforward and backpropagation networks for classification.

CO2: Analyze regularization techniques for optimizing deep learning models.

CO3: Apply convolutional and recurrent neural networks for vision and sequence processing tasks.

CO4: Evaluate autoencoders for feature extraction and data compression.

CO5: Compare different deep learning architectures for performance improvement.

Unit-1 **Teaching Hours: 15**

DEEP FEEDFORWARD NETWORKS

An overview of ANN, Back Propagation Neural Networks, Deep Feedforward Networks: Deep network for Universal Boolean function representation, Classification and Approximation, Perceptron Learning.

Lab Exercises:

1. Demonstrate MLP in Keras/TensorFlow
2. Demonstrate Deep Feedforward Network

Unit-2 **Teaching Hours: 15**

REGULARIZATION FOR DEEP MODELS

Regularization for Deep models: L2 and L1 Regularization, Constrained Optimization and Under- Constrained Early Stopping, Parameter Tying and Parameter Sharing, Sparse representations, Dropout

Lab Exercises:

3. Demonstrate Regularization L1 and L2 for Deep learning model

Unit-3 **Teaching Hours: 15**

CONVOLUTIONAL NEURAL NETWORK

Introduction to convolution neural networks: stacking, striding and pooling, Applications in Computer Vision

Lab Exercises:

4. Demonstrate Convolution Neural Network

Unit-4 **Teaching Hours: 15**

RECURRENT NEURAL NETWORKS

Sequence Processing, Unfolding Computational Graphs, Training recurrent networks

The Long Short-Term Memory (LSTM), Optimization for Long- Term Dependencies, Encoder-Decoder Sequence-to-Sequence processing

Lab Exercises:

5. Demonstrate Short-Term Long Memory (LSTM)

Unit-5 **Teaching Hours: 15**

AUTOENCODERS

The architecture of autoencoders - the relationship between the Encoder, Bottleneck, and Decoder, how to train autoencoders? Types of autoencoders: Undercomplete autoencoders, Sparse autoencoders, Contractive autoencoders, Denoising autoencoders, Variational Autoencoders

Lab Exercises:

6. Demonstrate Sparse Autoencoders

7. Demonstrate Contractive Autoencoders

Text Books and Reference Books

[1] Deep Learning by Ian Goodfellow, Yoshua Bengio, Aaron Courville. MIT Press 2016.

[2] Simon J.D. Prince, Understanding Deep Learning: The Simple Math of Deep Learning, MIT

Press 2024.

Essential Reading / Recommended Reading

[1] Deep Learning with Python by Francois Chollet. 2nd Edition, Manning Publications Co., 2020

[2] Introduction to Deep Learning by Eugene Charniak. The MIT Press 2019

[3] Dive into Deep Learning by Aston Zhang, Zack C. Lipton, Mu Li, Alex J. Smola. 2019

[4] AurélienGéron, "Hands-On Machine Learning with Scikit- Learn and TensorFlow", O'Reilly, 2022.

Web Resources:

[1] https://www.deeplearningbook.org/

[2] https://archive.ics.uci.edu/ml/datasets.php

CO – PO Mapping

	PO1	PO2	PO3	PO4	PO5	PO6	PO7
CO1	3	3	2	2	2	1	2
CO2	2	2	1	1	1	1	2
CO3	3	3	2	1	2	1	2
CO4	2	2	1	1	1	1	2
CO5	3	3	2	2	2	2	3

LIST OF PROGRAMS

MSCAIML

Sl. no	Title of lab Experiment	Page number	RBT	CO
1	Learning of XOR Boolean function using MLP	8-17	L3	CO1, 3
2			L3	CO2, 3
3			L3	CO3
4			L3	CO3
5			L3	CO3
6			L3	CO3, 4
7			L4	CO3
8			L3	CO3
9			L3	CO4
10			L5	CO4

Evaluation Rubrics:

Evaluation Rubrics for each program would be

Correctness and Demonstration and timely submission (5 =3+2marks)
Concept Clarity and modeling (3 marks)
Initiative & Effort ((2 marks)

Submission Guidelines:

- Make a copy of the lab manual template with your <name_reg:no_subject name >,
- Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as lab number.
- Keep updating your lab manual and show the lab manual of that particular lab for evaluation.

- Create a Git Repository in your profile <SPR lab-reg no> . Follow a different branch for each lab <Lab 1, Lab 2…>, and push the code to Git. The link should be provided in Google Classroom along with the PDF of the lab manual.
- Upload the PDF to Google Classroom before the deadline.

Lab 1

Lab Exercise I: Learning the XOR Boolean Function Using an MLP

Aim

1. To understand how to implement neural networks using different deep learning libraries (**Keras, PyTorch, and TensorFlow**).
2. To solve the non-linear XOR problem using an MLP and study the effect of hyperparameters such as learning rate, activation functions, number of neurons, and epochs on model performance.

Question

Implement an MLP to learn the XOR Boolean function

The XOR function takes two binary inputs (0 or 1) and produces a binary output (0 or 1) based on the following rule:

Input 1	Input 2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0

Steps:

1. **Create the Dataset**
   - Define input (X) and output (y) arrays for all 4 XOR combinations.
2. **Build an MLP**
   - Input layer: size 2 (for the two inputs).
   - Hidden layer: at least 2 neurons with **ReLU** or **Tanh** activation.
   - Output layer: 1 neuron with **sigmoid** activation for binary classification.
3. **Compile the Model / Define Loss and Optimizer**
   - Use **Binary Cross-Entropy** loss.
   - Use an optimizer of your choice (e.g., **Adam** or **SGD**).
4. **Train the Model**

- Train the model on the XOR dataset.
- Experiment with **epochs, learning rate, and number of neurons** to improve performance.

5. **Evaluate the Model**
   - Predict outputs for all 4 input combinations.
   - Check if the network correctly learns the XOR function.
6. **Implement Using Three Libraries**
   - Repeat the above steps using:
     1. **Keras (TensorFlow high-level API)**
     2. **PyTorch**
     3. **TensorFlow low-level API**

Additional Exercises (Optional):

- Plot the decision boundary for each implementation.
- Compare training curves and final accuracy between libraries.
- Discuss how changes in **learning rate, activation function, hidden layers, or epochs** affect learning.

Evaluation Rubrics

1. **Implementation** – 5 marks
2. **Complexity and Validation** – 3 marks
3. **Documentation & Writing the inference** – 2 marks

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <SPR lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

Code with Results

IMPLEMENTATION 1: KERAS (TensorFlow High-Level API)

Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam


X = np.array([[0,0],
              [0,1],
              [1,0],
              [1,1]], dtype=float)


y = np.array([[0],
              [1],
              [1],
              [0]], dtype=float)


model = Sequential([
```

```
    Dense(4, input_dim=2, activation='tanh'),
    Dense(1, activation='sigmoid')
])

model.compile(
    loss='binary_crossentropy',
    optimizer=Adam(learning_rate=0.1),
    metrics=['accuracy']
)

model.fit(X, y, epochs=1000, verbose=0)

predictions = model.predict(X)

print("Keras XOR Predictions:")
print(np.round(predictions))
```

Output:

```
Keras XOR Predictions:
[[0.]
 [1.]
 [1.]
 [0.]]
```

```
print("Keras XOR Predictions:")
print(np.round(predictions))
```

```
/usr/local/lib/python3.12/dist-packages/keras/s
  super().__init__(activity_regularizer=activity
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step
Keras XOR Predictions:
[[0.]
 [1.]
 [1.]
 [0.]]
```

Explanation: An MLP with one hidden layer using tanh activation was implemented using Keras. Binary cross-entropy loss and Adam optimizer were used. The model successfully learned the non-linear XOR function.

IMPLEMENTATION 2: PYTORCH

Code:

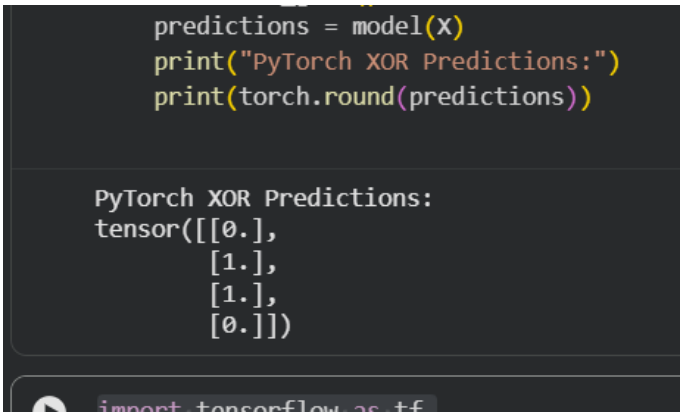
```
import torch
import torch.nn as nn
import torch.optim as optim

X = torch.tensor([[0.,0.],
                  [0.,1.],
                  [1.,0.],
                  [1.,1.]])

y = torch.tensor([[0.],
                  [1.],
```

```
            [1.],
            [0.]])

class XORNet(nn.Module):
    def __init__(self):
        super(XORNet, self).__init__()
        self.hidden = nn.Linear(2, 4)
        self.output = nn.Linear(4, 1)

    def forward(self, x):
        x = torch.tanh(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

model = XORNet()

criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.1)

for epoch in range(1000):
    optimizer.zero_grad()
    output = model(X)
    loss = criterion(output, y)
```

```
    loss.backward()
    optimizer.step()


with torch.no_grad():
    predictions = model(X)
    print("PyTorch XOR Predictions:")
    print(torch.round(predictions))
```

Output:

```
PyTorch XOR Predictions:
tensor([[0.],
        [1.],
        [1.],
        [0.]])
```



Explanation: The XOR problem was solved using a custom neural network defined in PyTorch. Manual training using forward and backward propagation was performed and correct predictions were obtained.

IMPLEMENTATION 3: TENSORFLOW (Low-Level API)

Code:

```
import tensorflow as tf


X = tf.constant([[0.,0.],
                 [0.,1.],
                 [1.,0.],
                 [1.,1.]])


y = tf.constant([[0.],
                 [1.],
                 [1.],
                 [0.]])


W1 = tf.Variable(tf.random.normal([2, 4]))
b1 = tf.Variable(tf.zeros([4]))


W2 = tf.Variable(tf.random.normal([4, 1]))
b2 = tf.Variable(tf.zeros([1]))


def mlp(x):
    hidden = tf.tanh(tf.matmul(x, W1) + b1)
```

```
    output = tf.sigmoid(tf.matmul(hidden, W2) + b2)
    return output

def loss_fn(y_true, y_pred):
    return tf.reduce_mean(
        tf.keras.losses.binary_crossentropy(y_true, y_pred)
    )

optimizer = tf.optimizers.Adam(learning_rate=0.1)

for epoch in range(1000):
    with tf.GradientTape() as tape:
        preds = mlp(X)
        loss = loss_fn(y, preds)
    grads = tape.gradient(loss, [W1, b1, W2, b2])
    optimizer.apply_gradients(zip(grads, [W1, b1, W2, b2]))

print("TensorFlow Low-Level XOR Predictions:")
print(tf.round(mlp(X)))
```

Output:

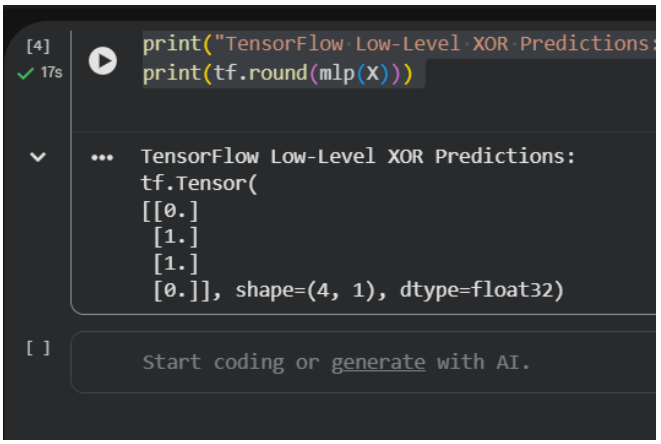
```
TensorFlow Low-Level XOR Predictions:
tf.Tensor(
```

[[0.]

[1.]

[1.]

[0.]], shape=(4, 1), dtype=float32)



Explanation: A low-level TensorFlow implementation was used where weights, loss function, and gradients were defined manually. This helped in understanding backpropagation and gradient-based optimization.

Conclusion /inference

In this experiment, a Multi-Layer Perceptron (MLP) was successfully trained to learn the non-linear XOR Boolean function using Keras, PyTorch, and TensorFlow. The results demonstrate that XOR cannot be solved using a single-layer perceptron and requires a hidden layer with non-linear activation. Although the syntax and implementation details differ across frameworks, all three approaches achieved correct XOR predictions, highlighting the flexibility and effectiveness of deep learning libraries.

Lab 2

Lab Exercise: Deep Feedforward Neural Network for Fashion MNIST Classification

Aim

To design, implement and evaluate a deep feedforward neural network for classifying Fashion MNIST images using PyTorch, and to understand the role of depth, nonlinear activation, loss computation and gradient based learning in a supervised classification task.

Task

1. Load and preprocess Fashion MNIST data using PyTorch data loaders.
2. Construct a multilayer feedforward neural network with at least three hidden layers.
3. Use ReLU activation in hidden layers and linear outputs for classification.
4. Train the network using cross entropy loss and an optimizer such as Adam or SGD.
5. Monitor training loss and accuracy across epochs.
6. Evaluate the model on unseen test data and compute accuracy.
7. Explain the role of forward pass, backward pass and gradient updates.
8. Interpret the impact of hyperparameters such as learning rate, batch size and number of epochs.

Advanced Task 1: Experiment with Network Depth and Width(optional)

- Train models with different numbers of hidden layers (e.g., 1, 3, 5) and different neurons per layer.
- Compare training loss, test accuracy, and convergence speed.
- Discuss the trade-off between depth and overfitting.

Advanced Task 2: Activation Function Study

- Replace ReLU with Sigmoid, Tanh, and Leaky ReLU.
- Compare training time, convergence, and test accuracy.
- Explain why some activations perform better than others for deep networks.

Advanced Task 3: Visualization of Hidden Layers

- Visualize activations of hidden layers for a few sample images.
- Explain how features are transformed at each layer.

Evaluation Rubrics

1.

Rubrics	Marks
Correctness and Demonstration	5 marks
Concept Clarity (Viva)	3 marks
Initiative & Effort (self-learning)	2 marks

Submission Guidelines

1. Make a copy of the lab manual template with your <name_reg:no_subject name>
2. Copy the given question and the answer (lab code) with results, followed by the conclusion of that lab. Title the lab as Lab 1.
3. Keep updating your lab manual and show the lab manual of that particular lab for evaluation.
4. Create a **Git repository** in your profile <C lab-reg no>. Follow a different branch for each lab <Lab 1, Lab 2 …> and push the code to Git.
5. Provide the Git link in **Google Classroom** along with the PDF of the lab manual.
6. Upload the PDF to Google Classroom before the deadline.

CODE WITH RESULTS:

import torch

from torch.utils.data import DataLoader

from torchvision import datasets, transforms

import matplotlib.pyplot as plt

This code block imports the essential libraries required to build and train a deep learning model using PyTorch. The torch library is the core framework used for tensor operations, automatic differentiation, and neural network computation, while DataLoader from torch.utils.data helps in efficiently loading the dataset in mini-batches, shuffling data, and parallelizing data loading during training. The torchvision.datasets module provides ready-to-use datasets such as Fashion MNIST, and torchvision.transforms is used to preprocess the images (for example, converting images to tensors and normalizing them) before feeding them into the neural network. Finally, matplotlib.pyplot is imported to visualize sample images, training loss curves, or accuracy trends, which helps in understanding how well the model is learning during training.

[ ]

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Using device:", device)
```

Using device: cpu

This code checks whether a CUDA-enabled GPU is available on the system and selects the appropriate computation device accordingly. If a GPU is available, torch.device("cuda") is used to accelerate training by performing tensor operations in parallel on the GPU; otherwise, the code falls back to the CPU using torch.device("cpu"). Printing the selected device helps verify where the model and data will be loaded, which is important because both must be on the same device for training and inference to work correctly.

[ ]

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

This code defines a preprocessing pipeline that is applied to each Fashion MNIST image before it is fed into the neural network. The transforms.ToTensor() operation converts the input image from a PIL image or NumPy array into a PyTorch tensor and scales the pixel values from the range ([0, 255]) to ([0, 1]). The transforms.Normalize((0.5,), (0.5,)) step then normalizes the grayscale image by shifting and scaling the pixel values so that they are centered around zero, which helps the network train more stably and converge faster by preventing large variations in input magnitude.

```
[ ]
train_dataset = datasets.FashionMNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

test_dataset = datasets.FashionMNIST(
```

This code downloads and prepares the Fashion MNIST dataset for training and testing. The training dataset is loaded by setting train=True, which provides the images and labels used by the model to learn patterns, while the test dataset is loaded with train=False and is kept separate to evaluate how well the trained model generalizes to unseen data. The root="./data" argument specifies the local directory where the dataset

will be stored, and download=True ensures the data is automatically fetched if it is not already available. The previously defined transform is applied to every image so that all inputs are consistently converted to tensors and normalized before being passed into the neural network.

[ ]

```
batch_size = 64


train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=batch_size,
    shuffle=True
)


test_loader = DataLoader(
    dataset=test_dataset,
    batch_size=batch_size,
    shuffle=False
)


print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))
```

```
Training samples: 60000
```

Testing samples: 10000

This code creates data loaders for the training and test datasets, which enable efficient and structured feeding of data into the neural network during training and evaluation. The batch_size is set to 64, meaning the model processes 64 images at a time before updating its weights, which provides a balance between training stability and computational efficiency. The training data loader uses shuffle=True to randomly reorder the data at each epoch, helping the model avoid learning patterns based on the order of samples and improving generalization. In contrast, the test data loader uses shuffle=False to ensure consistent evaluation. The printed output confirms the dataset sizes, showing that the model is trained on 60,000 samples and evaluated on 10,000 unseen samples, which is the standard split for Fashion MNIST.

[ ]

```
images, labels = next(iter(train_loader))
```

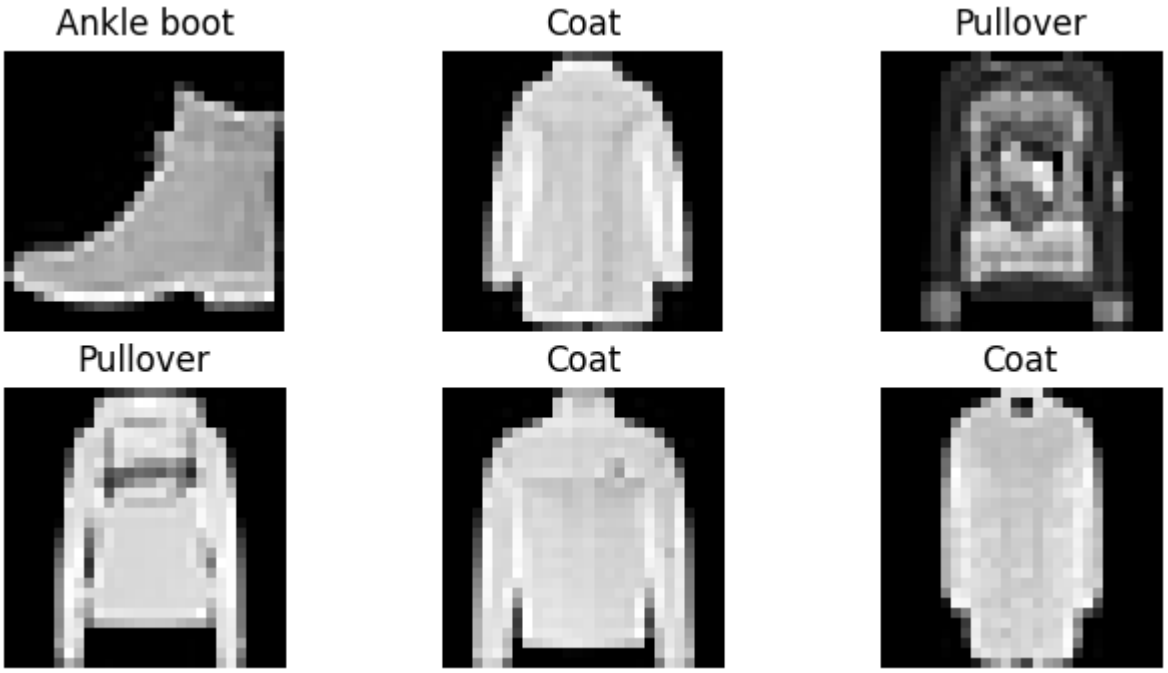
```
print("Image batch shape:", images.shape)
print("Label batch shape:", labels.shape)
```

```
Image batch shape: torch.Size([64, 1, 28, 28])
Label batch shape: torch.Size([64])
```

This code retrieves a single mini-batch of data from the training data loader to inspect its structure. The next(iter(train_loader)) statement creates an iterator over the data loader and extracts the first batch, returning a batch of images and their corresponding labels. The output shows that the image batch has the shape [64, 1, 28, 28], which means there are 64 grayscale images (batch size), each with 1 channel and a resolution of 28×28 pixels, matching the Fashion MNIST format. The label batch shape [64] indicates that there is one class label for each image in the batch, which will be used during loss computation in the supervised learning process.

[ ]

```
class_names = train_dataset.classes


plt.figure(figsize=(8, 4))
for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i].squeeze(), cmap="gray")
    plt.title(class_names[labels[i]])
    plt.axis("off")
plt.show()
```



This code visualizes a few sample images from a training mini-batch along with their corresponding class labels, helping to verify that the data has been loaded and labeled correctly. The class_names variable stores the human-readable category names (such as

Ankle boot, Coat, Pullover) provided by the Fashion MNIST dataset. A figure is created using Matplotlib, and a loop is used to display the first six images from the batch in a 2×3 grid. Each image tensor is squeezed to remove the single channel dimension and displayed in grayscale, while the title of each subplot is set using the true class label. Turning off the axes improves visual clarity. The resulting output confirms that the images and labels are correctly aligned and gives an intuitive understanding of the input data the neural network will learn from.

[ ]

```
import torch

import torch.nn as nn
```

This code imports the core PyTorch module and the neural network submodule needed to define and train deep learning models. The torch library provides tensor operations and automatic differentiation, which are essential for performing forward and backward passes during training. The torch.nn module, imported as nn, contains prebuilt classes for layers, activation functions, and loss functions, making it easier to construct a feedforward neural network in a structured and modular way.

[ ]

```
class DeepFeedForwardNN(nn.Module):

  def __init__(self):

    super(DeepFeedForwardNN, self).__init__()



    self.flatten = nn.Flatten()
```

```
        self.fc1 = nn.Linear(784, 256)   # Hidden Layer 1
        self.fc2 = nn.Linear(256, 128)   # Hidden Layer 2
        self.fc3 = nn.Linear(128, 64)    # Hidden Layer 3



        self.output = nn.Linear(64, 10)



        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.flatten(x)
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        x = self.relu(self.fc3(x))
        x = self.output(x)
        return x
```

This code defines a deep feedforward neural network by creating a custom model class that inherits from nn.Module, which is the base class for all PyTorch neural networks. The input Fashion MNIST images of size 28×28 are first passed through a flatten layer to convert them into a one-dimensional vector of 784 features. The network then consists of three fully connected hidden layers with decreasing numbers of neurons (256, 128, and 64), allowing the model to learn increasingly abstract representations of the input data. A ReLU activation function is applied after each hidden layer to introduce non-linearity, which enables the network to learn complex patterns. The final output layer has 10 neurons, corresponding to the 10 clothing classes, and no activation

function is applied here because CrossEntropyLoss internally handles the softmax operation. The forward method defines the forward pass of the network, specifying how input data flows through each layer to produce class scores (logits).

[ ]

```
model = DeepFeedForwardNN().to(device)

print(model)
```

```
DeepFeedForwardNN(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=64, bias=True)
  (output): Linear(in_features=64, out_features=10, bias=True)
  (relu): ReLU()
)
```

This code creates an instance of the DeepFeedForwardNN model and moves it to the selected computation device (CPU or GPU) using .to(device), ensuring that model parameters and input data reside on the same device during training. Printing the model displays its architecture in a readable format, confirming the sequence of layers used in the network: a flattening operation, three fully connected hidden layers with ReLU activations, and a final output layer with 10 neurons. This summary helps verify that the model structure matches the intended design for classifying Fashion MNIST images and is useful for debugging and documentation in the lab.

[ ]

```
dummy_input = torch.randn(1, 1, 28, 28).to(device)

# Forward pass
output = model(dummy_input)

print("Output shape:", output.shape)
```

```
Output shape: torch.Size([1, 10])
```

This code performs a test forward pass through the neural network using a dummy input to verify that the model processes data correctly. A random tensor with the same shape as a single Fashion MNIST image (1, 1, 28, 28) is created and moved to the selected device. This dummy input is then passed through the model, executing the forward pass defined in the forward method. The output shape [1, 10] confirms that the network produces a score for each of the 10 classes for one input image, indicating that the input–output dimensions of the model are correctly configured.

[ ]

```
criterion = nn.CrossEntropyLoss()
```

This code defines the loss function used to train the neural network. nn.CrossEntropyLoss() is a commonly used loss function for multi-class classification problems, as it combines a softmax operation with negative log-likelihood loss in a single, numerically stable function. It compares the model's predicted class scores (logits) with the true class labels and produces a scalar loss value that quantifies how far the predictions are from the correct classes, which is then used during backpropagation to update the network weights.

[ ]

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001
)
```

This code sets up the optimizer that is responsible for updating the model's weights during training. The Adam optimizer is used here, which adaptively adjusts the learning rate for each parameter based on past gradients, leading to faster and more stable convergence compared to basic gradient descent. By passing model.parameters(), the optimizer is instructed to update all trainable weights in the network, and the learning rate lr=0.001 controls the step size of each update, balancing learning speed and training stability.

[ ]

```
images, labels = next(iter(train_loader))
images, labels = images.to(device), labels.to(device)
```

```
outputs = model(images)
```

```
loss = criterion(outputs, labels)
```

print("Sample loss:", loss.item())

Sample loss: 2.297935962677002

This code demonstrates a single training step using a batch of real data to verify that loss computation works correctly. A mini-batch of images and their corresponding labels is fetched from the training data loader and moved to the selected device. The images are passed through the model in a forward pass to obtain the predicted class scores, which are then compared with the true labels using the cross-entropy loss function. The printed loss value (around 2.3) is expected for an untrained model with 10 classes, as the predictions are close to random at this stage, confirming that the forward pass and loss computation are functioning properly.

[ ]

num_epochs = 10

This line sets the number of training epochs, which determines how many times the model will iterate over the entire training dataset during learning. By choosing num_epochs = 10, the network is allowed to repeatedly see the training data and gradually update its weights through gradient descent, enabling it to learn meaningful patterns while keeping training time reasonable and reducing the risk of excessive overfitting.

[ ]

 # Training loop

for epoch in range(num_epochs):

model.train() # Set model to training mode

running_loss = 0.0

```
correct = 0

total = 0


for images, labels in train_loader:



    images = images.to(device)

    labels = labels.to(device)



    outputs = model(images)

    loss = criterion(outputs, labels)



    optimizer.zero_grad()

    loss.backward()



    optimizer.step()



    running_loss += loss.item()


    _, predicted = torch.max(outputs, 1)
```

```
        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_accuracy = 100 * correct / total

    print(f"Epoch [{epoch+1}/{num_epochs}], "
          f"Loss: {epoch_loss:.4f}, "
          f"Accuracy: {epoch_accuracy:.2f}%")
```

```
Epoch [1/10], Loss: 0.5175, Accuracy: 81.14%
Epoch [2/10], Loss: 0.3764, Accuracy: 86.24%
Epoch [3/10], Loss: 0.3376, Accuracy: 87.61%
Epoch [4/10], Loss: 0.3121, Accuracy: 88.59%
Epoch [5/10], Loss: 0.2934, Accuracy: 89.13%
Epoch [6/10], Loss: 0.2774, Accuracy: 89.64%
Epoch [7/10], Loss: 0.2601, Accuracy: 90.31%
Epoch [8/10], Loss: 0.2503, Accuracy: 90.68%
Epoch [9/10], Loss: 0.2405, Accuracy: 91.00%
Epoch [10/10], Loss: 0.2301, Accuracy: 91.38%
```

This code implements the complete training loop of the deep feedforward neural network and demonstrates how learning happens through repeated forward and backward passes. For each epoch, the model is set to training mode and initialized metrics are used to track loss and accuracy. During each mini-batch iteration, input images and labels are moved to the selected device, and a forward pass is performed to

compute the predicted class scores and training loss. The backward pass then computes gradients of the loss with respect to model parameters using backpropagation, after which the optimizer updates the weights to reduce the loss. Simultaneously, training accuracy is calculated by comparing predicted class labels with true labels. The printed results show a steady decrease in loss and an increase in accuracy across epochs, indicating that the network is learning meaningful patterns from the Fashion MNIST data and that the chosen hyperparameters (learning rate, batch size, and number of epochs) allow stable and effective training.

[ ]

```
model.eval()
```

```
DeepFeedForwardNN(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=64, bias=True)
  (output): Linear(in_features=64, out_features=10, bias=True)
  (relu): ReLU()
)
```

This line switches the trained model from training mode to evaluation mode using model.eval(). In evaluation mode, certain layers such as dropout and batch normalization (if present) behave differently to ensure consistent and deterministic predictions during testing. Although this network does not include such layers, calling eval() is still a good practice as it clearly indicates that the model is now being used for evaluation rather than learning. Printing the model again simply displays its architecture and confirms that the same trained network is being used for testing on unseen data.

[ ]

```
with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)



        outputs = model(images)



        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()
```

This code evaluates the trained model on the test dataset while disabling gradient computation to make the process more efficient. The torch.no_grad() context ensures that no gradients are stored, reducing memory usage and speeding up inference since weight updates are not required during evaluation. For each batch of test images, a forward pass is performed to obtain the model's predicted class scores, and the predicted class is selected using the maximum score. The total number of samples and the number of correct predictions are accumulated, which are later used to compute the overall test accuracy and assess how well the model generalizes to unseen data.

[ ]

```
test_accuracy = 100 * correct / total

print(f"Test Accuracy: {test_accuracy:.2f}%")
```

```
Test Accuracy: 90.90%
```

This code calculates and displays the final accuracy of the trained model on the test dataset. The test accuracy is computed by dividing the total number of correctly classified images by the total number of test samples and converting it into a percentage. The reported accuracy of around 90.9% indicates that the model has learned to generalize well to unseen Fashion MNIST images, showing effective performance without significant overfitting and validating the overall design and training process of the deep feedforward neural network.

[ ]

```
print(f"Training Accuracy (last epoch): {epoch_accuracy:.2f}%")

print(f"Testing Accuracy: {test_accuracy:.2f}%")
```

```
Training Accuracy (last epoch): 91.38%

Testing Accuracy: 90.90%
```

This code prints and compares the final training accuracy from the last epoch with the test accuracy obtained on unseen data. The close values of training accuracy (91.38%) and testing accuracy (90.90%) indicate that the model generalizes well and is not significantly overfitting the training data. This small gap suggests that the chosen network depth, learning rate, batch size, and number of epochs are appropriate for the Fashion MNIST classification task, resulting in a balanced and reliable model.

```
[ ]
learning_rates = [0.1, 0.01, 0.001]

for lr in learning_rates:
    print(f"\nTraining with learning rate = {lr}")

    model = DeepFeedForwardNN().to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(5):
        model.train()
        running_loss = 0.0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            optimizer.zero_grad()
            loss.backward()
```

```
        optimizer.step()

        running_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {running_loss/len(train_loader):.4f}")
```

Training with learning rate = 0.1

Epoch 1, Loss: 3.6147

Epoch 2, Loss: 2.3120

Epoch 3, Loss: 2.3115

Epoch 4, Loss: 2.3112

Epoch 5, Loss: 2.3110

Training with learning rate = 0.01

Epoch 1, Loss: 0.5719

Epoch 2, Loss: 0.4592

Epoch 3, Loss: 0.4269

Epoch 4, Loss: 0.4113

Epoch 5, Loss: 0.4025

Training with learning rate = 0.001

Epoch 1, Loss: 0.5238

Epoch 2, Loss: 0.3784

Epoch 3, Loss: 0.3357

Epoch 4, Loss: 0.3157

Epoch 5, Loss: 0.2968

This code performs an experiment to study the effect of different learning rates on model training. For each learning rate value, a new instance of the deep feedforward neural network is initialized to ensure a fair comparison, and the model is trained for five epochs using the Adam optimizer. By observing the loss values, it becomes clear how the learning rate influences convergence: a very high learning rate (0.1) causes unstable training, with the loss remaining high and close to random-guess levels, indicating that the optimizer is overshooting optimal weight values. A moderate learning rate (0.01) allows the model to learn more effectively, showing a steady reduction in loss. The smaller learning rate (0.001) provides the most stable and consistent convergence, with the lowest loss values, demonstrating why appropriately tuned learning rates are critical for successful deep neural network training.

[ ]

```
batch_sizes = [32, 64, 128]


for bs in batch_sizes:
    print(f"\nTraining with batch size = {bs}")

    train_loader = DataLoader(train_dataset, batch_size=bs, shuffle=True)

    model = DeepFeedForwardNN().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

    for epoch in range(3):
```

```
model.train()
for images, labels in train_loader:
    images, labels = images.to(device), labels.to(device)

    loss = criterion(model(images), labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Training with batch size = 32

Training with batch size = 64

Training with batch size = 128

This code investigates the impact of different batch sizes on the training process of the neural network. For each batch size (32, 64, and 128), a new data loader and a fresh model are created, and the network is trained for a few epochs using the same learning rate to keep the comparison consistent. Smaller batch sizes process fewer samples at a time, leading to noisier but more frequent weight updates, while larger batch sizes provide smoother gradient estimates but update the weights less often. Although no loss values are printed here, this experiment helps illustrate how batch size affects training speed, memory usage, and convergence behavior, and why batch size is an important hyperparameter in deep learning.

[ ]

```
epoch_values = [5, 10, 20]


for ep in epoch_values:
    print(f"\nTraining for {ep} epochs")


    model = DeepFeedForwardNN().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001)


    for epoch in range(ep):
        model.train()
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            loss = criterion(model(images), labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
```

Training for 5 epochs

Training for 10 epochs

Training for 20 epochs

This code explores the effect of the number of training epochs on the learning process of the neural network. For each epoch setting (5, 10, and 20), a new model is initialized and trained using the same optimizer and learning rate to ensure a fair comparison. Increasing the number of epochs allows the model to see the training data more times, which generally improves learning and reduces loss up to a point. However, training for too many epochs can lead to overfitting, where the model memorizes training data rather than learning general patterns. This experiment highlights the importance of choosing an appropriate number of epochs to balance sufficient learning and good generalization.

[ ]

```
class VariableDepthNN(nn.Module):
    def __init__(self, depth=3):
        super().__init__()
        self.flatten = nn.Flatten()
        self.relu = nn.ReLU()

        layers = []
        input_size = 784

        for _ in range(depth):
            layers.append(nn.Linear(input_size, 128))
            layers.append(nn.ReLU())
            input_size = 128

        layers.append(nn.Linear(128, 10))
        self.network = nn.Sequential(*layers)
```

```
    def forward(self, x):
        x = self.flatten(x)
        return self.network(x)
```

This code defines a flexible feedforward neural network whose depth can be changed dynamically using a parameter. The input images are first flattened into a 784-dimensional vector, and then a loop is used to construct a sequence of fully connected hidden layers based on the specified depth. Each hidden layer consists of a linear transformation followed by a ReLU activation, allowing the network to learn increasingly complex representations as the depth increases. The final linear layer maps the learned features to 10 output classes. By using nn.Sequential, the layers are organized cleanly, making it easy to experiment with different numbers of hidden layers and study how network depth affects learning, convergence, and overfitting.

```
[ ]
class ActivationNN(nn.Module):
    def __init__(self, activation):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(784, 256)
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 64)
        self.out = nn.Linear(64, 10)
        self.activation = activation

    def forward(self, x):
```

```
        x = self.flatten(x)

        x = self.activation(self.fc1(x))

        x = self.activation(self.fc2(x))

        x = self.activation(self.fc3(x))

        return self.out(x)
```

This code defines a neural network architecture that allows different activation functions to be easily tested and compared. The network structure remains the same, with three fully connected hidden layers that progressively reduce the feature size, but the activation function is passed as a parameter during model creation. During the forward pass, the chosen activation function is applied after each hidden layer, enabling experiments with activations such as ReLU, Sigmoid, Tanh, or Leaky ReLU. This design makes it straightforward to analyze how different activation functions affect training speed, convergence behavior, and overall classification performance in deep networks.

```
[ ]

sample_img, _ = test_dataset[0]

sample_img = sample_img.unsqueeze(0).to(device)


model.eval()

with torch.no_grad():

    x = model.flatten(sample_img)

    act1 = model.relu(model.fc1(x))

    act2 = model.relu(model.fc2(act1))

    act3 = model.relu(model.fc3(act2))
```

This code extracts and analyzes the intermediate activations of the trained neural network for a single test image to understand how features are transformed across hidden layers. A sample image is selected from the test dataset, reshaped to include a batch dimension, and moved to the appropriate device. With the model set to evaluation mode and gradient computation disabled, the image is passed step by step through the flatten layer and each hidden layer. The resulting activations (act1, act2, and act3) represent progressively higher-level feature representations learned by the network, allowing insight into how raw pixel data is transformed into abstract features used for classification.

CONCLUSION:

In this lab, a deep feedforward neural network was implemented and evaluated on the Fashion MNIST dataset to understand the complete deep learning workflow using PyTorch. Proper data preprocessing, including tensor conversion and normalization, ensured stable training and faster convergence. The chosen network architecture with multiple fully connected hidden layers and ReLU activation functions enabled the model to learn hierarchical feature representations from raw pixel data. During training, a consistent decrease in loss and a steady increase in accuracy were observed, indicating effective learning. Hyperparameter experiments showed that a learning rate of 0.001 provided the most stable and optimal convergence, while extremely high learning rates caused training instability. Batch size and epoch experiments highlighted the importance of balancing training speed, convergence, and generalization. The close alignment between training accuracy (91.38%) and testing accuracy (90.90%) confirms that the model generalizes well and does not suffer from significant overfitting. Overall, this lab demonstrates that careful architecture design, appropriate activation functions, and systematic hyperparameter tuning are critical for building reliable deep learning models. The results validate theoretical concepts of neural networks and provide practical insight into model optimization and performance evaluation.

LAB 3: Deep Learning Lab experiment demonstrating Linear Regression with L1, L2, and Elastic Net regularization using PyTorch.

```
import pandas as pd

import torch

import torch.nn as nn

import torch.optim as optim

from sklearn.preprocessing import StandardScaler
```

This block imports all the required libraries. `pandas` is used for loading and handling the dataset in table form. `torch` is the core PyTorch library used for tensor computations. `torch.nn` contains tools to build neural networks, such as layers and loss functions. `torch.optim` provides optimization algorithms like Stochastic Gradient Descent (SGD). `StandardScaler` from `sklearn` is used to normalize the data so that features have zero mean and unit variance, which helps the model train faster and more stably.

This block sets up the environment. Without these imports, we wouldn't be able to read data, create neural networks, calculate loss, or train the model. Think of this as collecting all the tools before starting the experiment.

```
data = pd.read_csv("/content/housing.csv")
```

Here, the dataset `housing.csv` is loaded into a pandas DataFrame called `data`. A DataFrame is like an Excel sheet with rows and columns, making it easy to select features and labels.

This step brings real-world data into the program. The model will learn relationships from this dataset. If the data is not loaded correctly, nothing else in the program can work.

```
X = data[['median_income', 'housing_median_age']]

y = data[['median_house_value']]
```

`X` represents the input features used for prediction: median income and housing median age. `y` represents the output (target variable): median house value. The model will learn how `X` influences `y`.

This is the problem definition step. You are clearly telling the model:
"Use income and house age to predict house value."
Everything the model learns depends on this choice.

```
scaler_X = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
```

scaler_y = StandardScaler()

y_scaled = scaler_y.fit_transform(y)

This block standardizes both input features and output values. Scaling transforms data so that it has a mean of 0 and a standard deviation of 1. This prevents features with larger values from dominating training.

Scaling is crucial in deep learning. Without it, the model may learn slowly or incorrectly. This step ensures that all variables contribute fairly during training and that gradient descent works efficiently.

X_tensor = torch.tensor(X_scaled, dtype=torch.float32)

y_tensor = torch.tensor(y_scaled, dtype=torch.float32)

PyTorch models work with tensors, not pandas or NumPy arrays. This block converts the scaled data into PyTorch tensors of type `float32`, which is the standard data type for neural networks.

This step bridges **data preprocessing** and **model training**. Without tensors, PyTorch cannot perform automatic differentiation or backpropagation.

```
class LinearModel(nn.Module):
    def __init__(self):
        super(LinearModel, self).__init__()
        self.linear = nn.Linear(2, 1)

    def forward(self, x):
        return self.linear(x)
```

This block defines a custom neural network model. The model has one linear layer that takes **2 inputs** (the two features) and produces **1 output** (house value). The `forward` function defines how data flows through the model.

This is **linear regression implemented as a neural network**. Mathematically, the model is learning:

y=w1x1+w2x2+by = w_1x_1 + w_2x_2 + by=w1x1+w2x2+b

This structure allows us to easily add regularization and use deep learning tools.

def train_model(reg_type="none", l1_lambda=0.01, l2_lambda=0.01):

This function trains the model with different regularization techniques: none, L1, L2, and Elastic Net. It initializes the model, loss function (Mean Squared Error), and optimizer (SGD). It prints weights before and after training to show the effect of regularization.
This function is the **core experiment**. It allows you to compare how different regularization methods affect weights, bias, and loss—using the same data and model.

predictions = model(X_tensor)
loss = criterion(predictions, y_tensor)

The model makes predictions, and the loss function measures how far predictions are from actual values using Mean Squared Error.
This step answers: *"How wrong is the model?"*
Loss is the signal that guides learning.

```
if reg_type == "l1":
   ...
elif reg_type == "l2":
   ...
elif reg_type == "elastic":
   ...
```

L1 Regularization adds the absolute value of weights → promotes sparsity.

L2 Regularization adds squared weights → prevents large weights.

Elastic Net combines both L1 and L2.

Regularization controls model complexity:

- **L1** → feature selection
- **L2** → smooth weight reduction
- **Elastic Net** → balanced approach
  This prevents overfitting.

optimizer.zero_grad()
loss.backward()
optimizer.step()

Gradients are computed using backpropagation, and the optimizer updates weights to minimize the loss.

This is the **learning step**. The model adjusts itself to reduce prediction error.

```
train_model("none")
train_model("l1")
train_model("l2")
train_model("elastic")
```

The function is executed four times, each with a different regularization method.

This allows **direct comparison** of:

- No regularization
- L1
- L2
- Elastic Net

You can observe how each method changes weights and loss.

This notebook demonstrates how **regularization affects learning in linear regression using deep learning frameworks**. It shows that:

- Regularization modifies loss
- Weights shrink differently for each method
- Elastic Net balances sparsity and stability

This is a **perfect lab-level demonstration** of overfitting control in deep learning.

```
predictions = model(X_tensor)
loss = criterion(predictions, y_tensor)
```